UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

# Calidad de Software

Mag. Pablo Virgolini

31 de agosto de 2026

# Caso: Knight Capital (2012)

**▶ Intro**

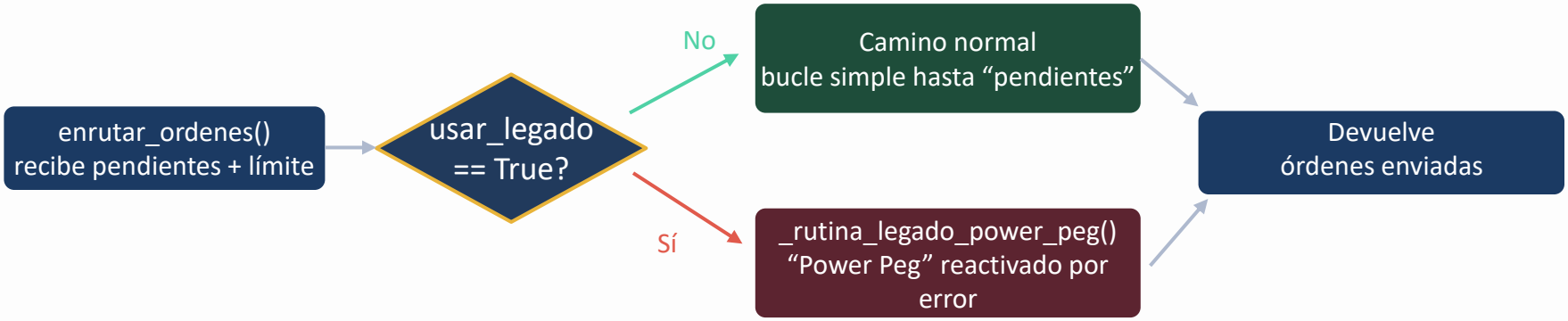
Te invito a leer el caso de la Knight Capital (2012).

**▶ Mecanismo del Bug**

Para incorporar una nueva funcionalidad en su sistema de enrutamiento de órdenes, Knight Capital debía actualizar el software en ocho servidores. Sin embargo, el despliegue se realizó correctamente solo en siete. El servidor que quedó sin actualizar conservaba una versión anterior del sistema y, al recibir una bandera reutilizada para la nueva funcionalidad, activó por error una rutina antigua que todavía permanecía en producción. Como consecuencia, comenzó a generar órdenes bursátiles incorrectas de manera automática y a gran velocidad.

# Código simulado

El Código simula un enrutador de órdenes simplificado: recibe órdenes pendientes de un cliente y las envía, respetando (en teoría) un límite diario que fija la firma.




**Importante:** ninguno de los dos caminos compara la cantidad enviada contra el límite diario mientras envía. El parámetro `limite_diario` se recibe, pero nunca se usa para cortar el envío.

# enrutar_ordenes()

```
def enrutar_ordenes(ordenes_pendientes, limite_diario,
                    usar_legado=False):
    if usar_legado:
        return _rutina_legado_power_peg(
                    ordenes_pendientes)

    enviadas = 0
    while enviadas < ordenes_pendientes:
        enviadas += 1
    return enviadas

# limite_diario nunca aparece
# condición del while  ← ese es el bug
```

## Lo que debería hacer

- Enviar órdenes una por una, igual que ahora.
- Detenerse en cuanto se alcance limite_diario, sin importar cuántas queden pendientes.
- Comportarse igual en el camino normal y en el legado.

**Lo que hace en realidad:** ignora limite_diario en ambos caminos.

# _rutina_legado_power_peg()

```
def _rutina_legado_power_peg(cantidad_objetivo):
    ordenes_enviadas = 0
    while ordenes_enviadas < cantidad_objetivo:
        ordenes_enviadas += 1
    return ordenes_enviadas

# No conoce el concepto de "limite_diario".
# Nunca lo necesitó: otro componente lo controlaba y ya
no existe.
```

**Paralelo con Knight Capital:** esta función representa a "Power Peg", la rutina que ya no debía usarse pero seguía en el binario, lista para activarse por una bandera mal configurada.

# Comportamiento observado: salidas

**Ejemplo 1**

```
enrutar_ordenes(pendientes=212, limite=250)
```

**212**

dentro del límite

**Ejemplo 2**

```
enrutar_ordenes(pendientes=500, limite=250)
```

**500**

⚠ supera el límite de 250

**Ejemplo 3 (legado)**

```
enrutar_ordenes(pendientes=500, limite=250,
usar_legado=True)
```

**500**

⚠ supera el límite de 250

El programa nunca lanza una excepción: sencillamente hace lo que el bucle le dice, sin ninguna verificación de negocio. Que “corra” no significa que sea correcto.

# Comportamiento observado: tests

| Test | Qué verifica | Resultado |
|---|---|---|
| `test_no_supera_lo_pedido_si_hay_margen` | Normal: 100 pendientes, límite 250 | **OK** |
| `test_respeta_el_limite_diario` | 500 pendientes, límite 250 → debería ser ≤ 250 | **FALLA (da 500)** |
| `test_legado_tambien_respeta_el_limite_diario` | Mismo caso, camino legado | **FALLA (da 500)** |
| `test_valor_limite_ordenes_pendientes_cero` | 0 pendientes → 0 enviadas | **OK** |
| `test_valor_limite_limite_diario_cero` | límite=0 → debería enviar 0 | **FALLA (da 100)** |

**2 pasan / 3 fallan.** Cada test que falla es un defecto real de la regla de negocio “nunca superar el límite diario” — no un error de sintaxis.

# ¿Avanzamos?



# ¿Qué es una métrica de calidad?

# Métrica de Calidad

Es una medición cuantitativa de un atributo del software, del proceso que lo produce o del proyecto que lo gestiona. Convierte una percepción en un dato comparable y verificable.

## Evaluación subjetiva

- “Este módulo parece difícil de mantener”
- “Este código se ve prolijo”
- Depende de la experiencia y el criterio de quien opina
- Difícil de comparar entre proyectos o en el tiempo

## Evaluación objetiva (métrica)

- “Este módulo tiene complejidad ciclomática 24”
- “Este archivo tiene 18% de líneas duplicadas”
- Se calcula de la misma forma, siempre
- Permite comparar, hacer seguimiento y fijar umbrales

# Uso de las métricas

Las métricas no son un fin en sí mismas: su valor está en las decisiones que permiten tomar.



## Diagnóstico

Identificar en qué parte del sistema está concentrado el riesgo o la mala calidad, antes de que se convierta en un incidente.



## Toma de decisiones

Priorizar qué refactorizar, dónde reforzar las pruebas o si un módulo está listo para pasar a producción.



## Comparación

Contrastar equipos, versiones o proyectos con una misma vara, en lugar de opiniones informales.



## Mejora continua

Fijar objetivos concretos (“bajar la duplicación del 15% al 5%”) y verificar el progreso en el tiempo.

# Tipo de métricas

## De producto

Miden características del software en sí: tamaño, complejidad, cobertura de pruebas, densidad de defectos.

Ej.: complejidad ciclomática de un módulo.

## De proceso

Miden cómo se desarrolla el software: duración de las fases, frecuencia de integración, tiempo de corrección de errores.

Ej.: tiempo promedio entre detección y corrección de un bug.

## De proyecto

Miden la gestión del proyecto en su conjunto: costos, plazos, tamaño del equipo, productividad.

Ej.: desvío entre el cronograma planificado y el real.

# Métricas producto

**Es el examen del código fuente** sin ejecutar el programa: una herramienta (o una persona) recorre el código, construye una representación interna de su estructura y aplica reglas para detectar patrones problemáticos.

## Análisis estático

- Se hace sobre el código fuente
- No requiere ejecutar el programa
- Puede correr en cada commit o pull request
- Encuentra problemas antes de llegar a pruebas dinámicas

## Análisis dinámico

- Requiere ejecutar el programa
- Es lo que hacen las pruebas unitarias, de integración, etc.
- Depende de los casos de prueba diseñados
- Encuentra problemas de comportamiento en tiempo de ejecución

# Métricas de código

Dentro de las métricas de producto, las métricas de código son las que se calculan directamente sobre el código fuente:



## Líneas de código (LOC)

Tamaño del software en cantidad de líneas



## Complejidad ciclomática

Cantidad de caminos independientes en el código



## Acoplamiento (coupling)

Grado de dependencia entre módulos



## Duplicación de código

Porcentaje de código repetido en el sistema



## Cobertura y densidad de defectos

% de código probado y defectos por unidad de tamaño

# Métricas de código: Línea de Código

Cantidad de líneas de código fuente de un programa. Es la métrica de tamaño más simple y más antigua, base de muchas otras métricas (defectos por LOC, costo por LOC).

**Ejemplo**

```
public class ItemPedido {
  private final Producto producto;
  private final int cantidad;

  public ItemPedido(Producto p, int c) {
    this.producto = p;
    this.cantidad = c;
  }
  // ...getters + getSubtotal()
}

# ItemPedido.java: ~18 líneas
# (de las clases más chicas)
```

Es fácil de calcular pero poco confiable por sí sola: no distingue código simple de código complejo, ni buen diseño de malo.

Sí sirve para ubicar rápidamente dónde se concentra el tamaño: en CalidadPedidos, once clases suman ~360 líneas, pero una sola clase concentra más de una quinta parte de ese total.

# Métrica de código: complejidad ciclomática

Propuesta por Thomas McCabe (1976). Mide la cantidad de caminos de ejecución independientes en una función, contando sus decisiones (if, for, while, case, and/or).

**Ejemplo**

```
def clasificar(nota):
    if nota >= 9:        # +1
        return "A"
    elif nota >= 7:      # +1
        return "B"
    elif nota >= 4:      # +1
        return "C"
    else:
        return "D"

# Complejidad ciclomática = 4
# (1 base + 3 decisiones)
```

1-10: función simple, bajo riesgo.
11-20: moderadamente compleja.
21-50: compleja, alto riesgo, difícil de testear.
>50: prácticamente imposible de probar exhaustivamente.

Cuanto mayor la complejidad, más casos de prueba se necesitan para cubrir todos los caminos.

# Métrica de código: acoplamiento

Mide el grado de dependencia entre módulos, clases o componentes. Un acoplamiento alto significa que un cambio en un módulo probablemente obligue a modificar otros.

**Ejemplo**

```
public class ServicioPedidos {
  private final RepositorioPedidos repositorio;
  private final ValidadorPedido validador;
  private final CalculadorDescuentos
                calculador;
  private final ReportePedidos reporte;

  public ServicioPedidos(...) { ... }

# ServicioPedidos conoce 4 clases
# concretas: ese es su acoplamiento
```

Acoplamiento alto → cambios en cascada, mayor costo de mantenimiento, más difícil de testear en forma aislada.

Aquí las dependencias llegan por constructor (inyección de dependencias), lo que ayuda a probar ServicioPedidos de forma aislada, aunque el número de clases conocidas siga siendo alto.

# Métrica de código: duplicación de código

Porcentaje de líneas o bloques de código que están repetidos (idénticos o casi idénticos) en distintas partes del sistema, en lugar de estar centralizados en un único lugar.

**Ejemplo**

```
calcularDescuento(cliente, pedido) {
  if (cliente.isVip()) {
    if (total > 1_000_000) ...
    // cascada de ifs anidados
  } ...
}

calcularDescuentoPromocional(
    cliente, pedido) {
  // misma cascada de ifs anidados,
  // copiada casi sin cambios
}
```

En CalculadorDescuentos, dos métodos comparten casi toda su lógica de decisión: la misma cascada de ifs aparece copiada, con solo la parte final distinta.

Cada copia duplicada es un lugar más donde corregir el mismo bug. La refactorización (extraer esa lógica común a un solo método) es la forma habitual de reducirla.

# Métrica de código: cobertura y densidad de defectos

Cobertura de pruebas: porcentaje de líneas, ramas o condiciones del código que son ejecutadas por la suite de tests automatizados.

Densidad de defectos: cantidad de defectos encontrados en relación al tamaño del código (por ejemplo, defectos cada 1000 LOC).

**Ejemplo**

$$\text{Cobertura de líneas} = \frac{\text{líneas ejecutadas por tests}}{\text{líneas totales}} \times 100$$

$$\text{Densidad de defectos} = \frac{\text{cantidad de defectos}}{\text{KLOC (miles de líneas)}} \times 1000$$

Una cobertura alta no garantiza ausencia de defectos (se puede ejecutar una línea sin verificar realmente su resultado).

En CalidadPedidos, solo 2 de las 11 clases (CalculadorDescuentos y ValidadorPedido) tienen tests con JUnit. El resto no tiene ningún test.

# Ramas muertas y condiciones redundantes

No todo camino muerto es un error de sintaxis: a veces hay que leer la lógica para notarlo. Ejemplo real, de ValidadorPedido en CalidadPedidos:

**Ejemplo (ValidadorPedido.java)**

```
if (pedido.getTotal() < 0) {
    return false;
}

// ...otras validaciones...

if (pedido.getTotal() < 0) {
    System.out.println("...");
    return false;
}
// ↑ esta segunda condición nunca
// puede ser verdadera: si el total
// fuera negativo, ya se habría
// retornado más arriba.
```



### ¿Por qué es una rama muerta?

La primera condición ya cubrió y resolvió el caso “total negativo”. Para cuando el flujo llega a la segunda, ese camino es lógicamente imposible.



### ¿Cómo se detecta sin ejecutar?

Alcanza con leer la secuencia de condiciones: no hace falta correr el programa para razonar que la segunda nunca se cumple. Eso es análisis estático.

# Herramientas y tecnologías

El análisis estático no se hace a mano: se automatiza con herramientas que corren en el editor, en cada commit o en el pipeline de integración continua.



## SonarQube

Plataforma de referencia para análisis estático continuo: calcula métricas de código, detecta bugs, vulnerabilidades y code smells.

### ESLint / Pylint

Linters de propósito general para JavaScript y Python: estilo, errores comunes y buenas prácticas.

### Checkstyle / PMD

Verifican convenciones de código y detectan patrones riesgosos en proyectos Java.

### Coverity / CodeQL

Motores de análisis más profundos, orientados a seguridad y detección de vulnerabilidades complejas.

CASO DE ESTUDIO



# Calidad de Pedidos

Vamos a analizar con métricas y SonarQube un Proyecto desarrollado en Java.

# Caso: Sistema para Gestión de pedidos

▶Un sistema Java (Maven) generado con IAG que registra clientes y productos, arma pedidos, aplica descuentos y genera reportes. Está operativo y tiene algunas pruebas automatizadas, pero nunca fue evaluado con métricas ni con análisis estático

▶ **Estructura**

```
ar.edu.calidadpedidos/
  Main.java
  Cliente.java
  Producto.java
  Pedido.java
  ItemPedido.java
  ValidadorPedido.java
  CalculadorDescuentos.java
  ServicioPedidos.java
  RepositorioPedidos.java
  ReportePedidos.java
  ConfiguracionConexion.java

src/test/.../ (JUnit 5)
  CalculadorDescuentosTest.java
  ValidadorPedidoTest.java
```

# Ejercicio: complejidad ciclomática real

Calculen a mano la complejidad ciclomática de calcularDescuento(), el método más grande de CalidadPedidos. Después lo van a poder comparar con lo que reporte SonarQube.

```
double calcularDescuento(
    Cliente cliente, Pedido pedido) {
  double descuento = 0;
  double total = pedido.getTotal();
  if (cliente != null) {
    if (cliente.isVip()) {
      if (total > 1_000_000) desc = .20;
      else if (total > 500_000) desc=.15;
      else if (total > 100_000) desc=.10;
      else descuento = 0.05;
    } else {
      if (total > 1_000_000) desc = .08;
      else if (total > 500_000) desc=.05;
      else if (total > 100_000) desc=.03;
    }
  }
  if (cliente != null
      && cliente.getCategoria() ==
         "MAYORISTA") descuento += .02;
  return total * descuento;
}
```

## Consignas

- Calculen la complejidad ciclomática paso a paso (1 base + cada decisión, incluyendo el &&)
- ¿En qué rango de riesgo cae (bajo, moderado, alto)?
- Este método tiene un “gemelo” casi idéntico en la misma clase: ¿qué otra métrica salta a la vista?
- Propongan una forma de reestructurar la cascada de ifs para bajar la complejidad

# Actividad: análisis con SonarQube

Trabajo en grupo: 4 integrantes. Fecha de presentación: 7/sept/2026

### 1. Exploración inicial

Ejecutar el sistema y los tests. Dar una primera valoración (alta/media/baja) SIN mirar SonarQube todavía.

### 2. Métricas con SonarQube

Correr el análisis estático y registrar las métricas relevantes: tamaño, complejidad, duplicación, cobertura, fiabilidad, seguridad, mantenibilidad.

### 3. Interpretar ≥4 métricas

No alcanza con el valor: hay que explicar dónde se concentra y qué consecuencias tiene.

### 4. Analizar 3 hallazgos

Clase/método afectado, el problema, por qué importa, una mejora posible y su prioridad.

### 5. Flujo de control

Encontrar al menos un caso de condición redundante o código inalcanzable, y justificarlo sin ejecutar el programa.

### 6. Producto / proceso / proyecto

Clasificar las métricas obtenidas y proponer una métrica de proceso y una de proyecto que las complementen.

### 7. Priorizar

Elegir 3 aspectos para intervenir antes de la próxima versión, fundamentando con riesgo, impacto y esfuerzo.

### 8. Reflexión final

¿Un programa que ejecuta bien y pasa sus tests tiene necesariamente buena calidad interna?

# Entregable y criterios de evaluación

**Entregable:** informe en PDF con evidencia de ejecución (captura), las métricas elegidas, el análisis de los 3 hallazgos, la priorización, la mejora aplicada, la comparación antes/después y las conclusiones. Se entrega junto con el código fuente modificado.

| |
|---|
| Ejecución y obtención correcta de métricas |
| Interpretación de las métricas |
| Análisis de hallazgos y flujo de control |
| Priorización y toma de decisiones |
| Implementación y evaluación de la mejora |
| Reflexión crítica y claridad del informe |